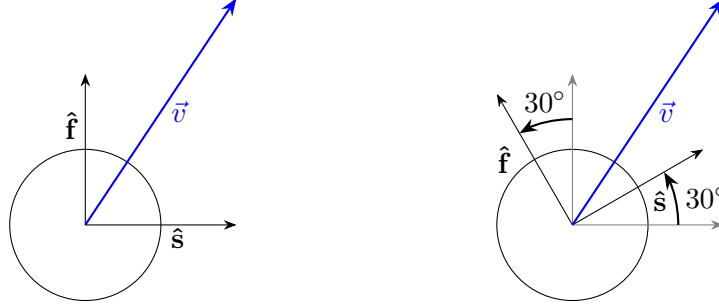


1 Determining velocity in reference frame of vessel

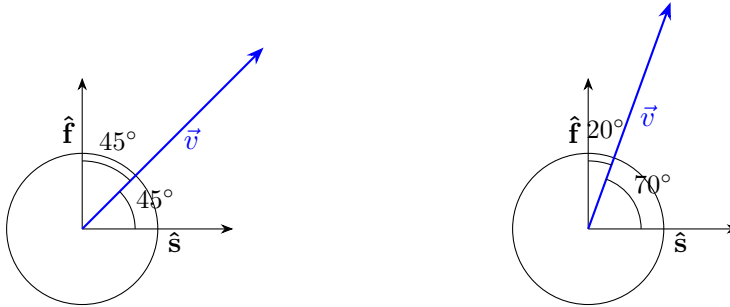
Vector $\vec{v}$ is our velocity. Vectors $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$ are forward and starboard directions of a vessel. So $\hat{\mathbf{f}}$ would be the 'nose'. Think of these as orientation vectors.



In the reference frame of the ground (or planet) $\vec{v}$ does not change direction as the vessel rotates. Notice in the drawing $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$ rotate 30 degrees. But $\vec{v}$ remains static. (Imagine the illustrations as a top down view from an external observer.)

As $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$ are unit vectors, they will have a magnitude of 1. While $\vec{v}$ will have a variable magnitude representing groundspeed in m/s. The goal is to calculate $\vec{v}$ but represented in $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$.

A dot product tells us how much two vectors point in the same direction. Imagine walking along $\vec{v}$ at a 45° , you will walk equally along the vectors $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$. If we change the angle of $\vec{v}$ to 20° from $\hat{\mathbf{f}}$, then we will walk further along $\hat{\mathbf{f}}$ than $\hat{\mathbf{s}}$.



Because of this relation, we can simply do as following:

$$\begin{aligned} \text{Forward velocity} \\ V_f &= \hat{\mathbf{f}} \cdot \vec{v} \\ \text{Starboard velocity} \\ V_s &= \hat{\mathbf{s}} \cdot \vec{v} \end{aligned}$$

Because of the properties of the dot product, if $\vec{v}$ points opposite of $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$, the resulting dot product will be negative. This negative value represents a velocity pointing opposite to our orientation vectors.

2 Translating relative velocity to counteractive control inputs

So far we have thought of $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$ as isolated orientation vectors in a 2d space. However, our vessel flies in a 3d space. In the code we simply remove the "up" element.

The means of how we determine the forevector is important. The vessel this system is made for is designed such that, when it is neutral, the navball will say we are pointing straight up. So if we just got the forevector though kOS' structures, we would get a vector that points straight up. This is obviously in conflict with how we represent $\hat{\mathbf{f}}$ and $\hat{\mathbf{s}}$.

The forevector of a vessel in kOS is always where the navball is pointing. This means that by default, the forevector is equal to the vessels Z-axis. KOS allows us to easily get the top-axis, which is the Y-axis. This axis is also the same as the direction the craft will rotate, if we pitch down (press W).

In our code, this looks as follows

```
function calc_forward_vector {
    return ship:facing:topvector.
}

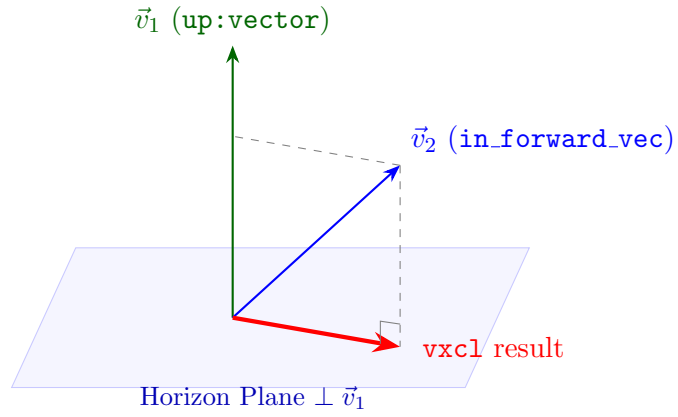
function calc_foward_hor_vector {
    parameter in_forward_vec.
    return vxcl(up:vector, in_forward_vec):normalized.
}
```

As you can see, we have two functions. One that calculates (returns) the forward (top) vector, and one that returns the foward vector, without the components of the up vector. Note, that the up vector is up relative to the frame of the planet. So really we are removing the (unit) vector that points from the center of the planet, to our vessel. We use the inbuilt function vxcl (vector exclude) to accomplish this.

kOS explains how vxcl works as follows:

*This is a vector, v2 with all of v1 excluded from it.
In other words, the projection of v2 onto the plane that is normal to v1.*

(Parameters v1 being up:vector and v2 being in_forward_vec.)



Because we are projecting onto the plane, and the forward vector often will be at an angle as the vessel pitches and yaws, the result won't be a unit vector. To fix this we use the `:normalized` function and restore it to a unit vector.